INFOPHARMA ORDERING SYSTEM (IPOS)

Server Application Subsystem

Summary

Acting as a formal proposal and blueprint for the InfoPharma Ordering System. This document details the architecture and requirements for the Server Application (IPOS-SA) subsystem.

Author

Engine Room

Contents

Preface 2
1.1 Purpose and scope of documentation 2
1.2 2
1.3 2
2. Introduction 3
3 Glossary 4
4 User Requirements 5
5. System requirements 6
5.1 Purpose and scope of the system 6
5.2 System Models 6
5.2.1 UML use-case models 6
5.2.1.1 Use case diagram 6
5.2.1.2 Use case specifications (simplified) 7
5.2.1.3 Prioritised List of use cases 36
6. Anticipated Evolution of System Requirements 39
6.1 Integration with Online Banking 39
6.2 Enterprise IT Administration 39
7 High Level Design 39
7. 1 IPOS-SA (Server Application) 39
7.2 IPOS-CA (Client Application) 39
7.3 IPOS-PU (Public Online Platform) 40
7.4 Software Architecture 40
7.5 Source Code for the Subsystem Interfaces 42
7.6 Testing the communications between subsystems 48
8. Appendices 51
8.1 Initial Statement of Requirements by the Client 51
8.2 Hardware (Minimal and Optimal Configuration) 52
8.2.1 Minimal hardware configuration 52
8.2.2 Optimal hardware configuration 52

Preface

1.1 Purpose and scope of documentation

This document consists of requirement specification and high-level design of one of the subsystems in InfoPharma Ordering System-Server Application(IPOS-SA), which the other two subsystems are InfoPharma Ordering System-Client Application (IPOS-CA) and InfoPharma Ordering System Online Platform (IPOS-PU). The document defines the functional and non-functional requirements of the system and outlines the overall architecture of the IPOS solution. The document is provided in response to the initial statement of requirements by InfoPharma ltd, the client.

The scope of this document includes the description of system behaviour, system indications, and how the subsystems integrate with the other IPOS subsystems. It focuses on analysing the business needs of InfoPharma ltd and translating them into the system requirements.

This document does not provide detailed implementation or low-level design of how IPOS-SA will be implemented. Various UML models will be used with fragments of software code to convey in more detail to system developers the expected behaviour of IPOS subsystem and how this subsystem will interact with other subsystems of IPOS.

1.2

This document if for:

- The managing directors and stakeholders of InfoPharma ltd
- The system analyst and architect of Engine Room.
- Testers and project managers responsible for validating and delivering the system.

Stakeholders and management should focus on the user and system requirements sections to understand the business value and functionality. Technical staff should review the high-level design, system architecture, and models to guide development and integration.

1.3

This is a new document, first version, v 1.0.

2. Introduction

This document shows the Requirements Specification and High-Level Design for the IPOS-SA subsystem, developed by Team A as part of the Integrated Pharmaceutical Ordering System (IPOS). The purpose of this document is to explain in depth what the IPOS-SA subsystem is responsible for, how it is structured, and how it will interact with the other subsystems within IPOS. It explains the main functionality provided by the subsystem and presents the design decisions that guide its implementation.

The IPOS system is made up of three subsystems: IPOS-SA (Server Administration), IPOS-CA (Client Application), and IPOS-PU (Public User). The IPOS-SA subsystem acts as the central server component of the system. It manages the core business operations related to wholesale pharmaceutical ordering. This includes authenticating merchants, managing the product catalogue, monitoring stock levels, processing wholesale orders, tracking order status, and calculating outstanding balances. In addition, IPOS-SA is responsible for handling commercial applications and supporting payment and email services through defined interfaces.

Each subsystem in IPOS maintains its own separate database. As a result, no subsystem directly accesses another subsystem's data. Instead, communication between subsystems takes place through clearly defined Java interfaces. This approach ensures a clean separation of responsibilities and makes the system easier to maintain and extend. By defining clear service boundaries, each team can develop its subsystem independently while still ensuring proper integration.

The overall goal of the IPOS-SA design is to provide a structured and modular server-side component that supports the wider IPOS system. The subsystem has been designed in a way that allows gradual development and testing. Interfaces are defined first, and implementation can then be built around them. This helps ensure consistency between the design models and the final code.

The sections that follow describe the detailed requirements of the subsystem, the architectural design, the interface definitions, and the generated source code. Additional supporting information is included in the appendices.

3 Glossary

Merchant
An authorised business user (e.g., pharmacy) that places wholesale orders through the IPOS system.

Commercial Application
A request submitted by a company seeking approval to become a registered commercial member within the IPOS system.

Application
A data object representing the details submitted for commercial registration, including company information.

ApprovalStatus
An enumeration representing the outcome of a commercial application (e.g., approved, rejected, pending).

Product
A pharmaceutical item available in the wholesale catalogue. A product includes identifying information such as product ID, name, and pricing details.

Stock
The quantity of a specific product currently available within the system.

StockItem
A data representation of a product and its associated stock level.

Order
A wholesale request submitted by a merchant containing one or more products and associated quantities.

OrderItem
An individual product entry within an order, specifying the product and requested quantity.

OrderStatus
An enumeration representing the current state of an order (e.g., accepted, processing, dispatched, delivered, cancelled).

OrderConfirmation
A response returned after a successful order placement, containing order reference details.

Outstanding Balance
The total amount owed by a merchant for previously processed orders.

Authentication
The process of verifying a merchant's identity using credentials such as username and password.

Payment Service
An abstracted service responsible for handling financial transactions related to orders.

Email Service
An abstracted service responsible for sending notifications such as order confirmations or application results.

Interface
A Java construct that defines a contract of methods that must be implemented by a class. Interfaces are used to define subsystem boundaries and integration points.

Subsystem
A distinct functional component within the IPOS system that operates independently with its own database and responsibilities.

UML

Unified Modelling Language

Use case

A basic interaction of a system with an actor.

4 User Requirements

THIS SECTION IS LEFT BLANK, SO WE DONT NEED TO INCLUDE IT IN OUR DOCUMENT

5. System requirements

5.1 Purpose and scope of the system

This section provides a more detailed specification of the functionality required for the InfoPharma Ordering System – Server Application, also known as the IPOS-SA.

This section is intended for the people responsible for the implementation of the IPOS, namely the system architects, designers and software engineers. We are using UML to create a detailed and high-level design, so Mr Lancaster might not immediately understand the diagrams and specification, but we encourage him to review and bring up any questions he may have about it if possible.

5.2 System Models

5.2.1 UML use-case models

5.2.1.1 Use case diagram

The use case diagram for the IPOS-SA subsystem is shown below. Other subsystems of IPOS that interact with the Server Application are treated as external systems.

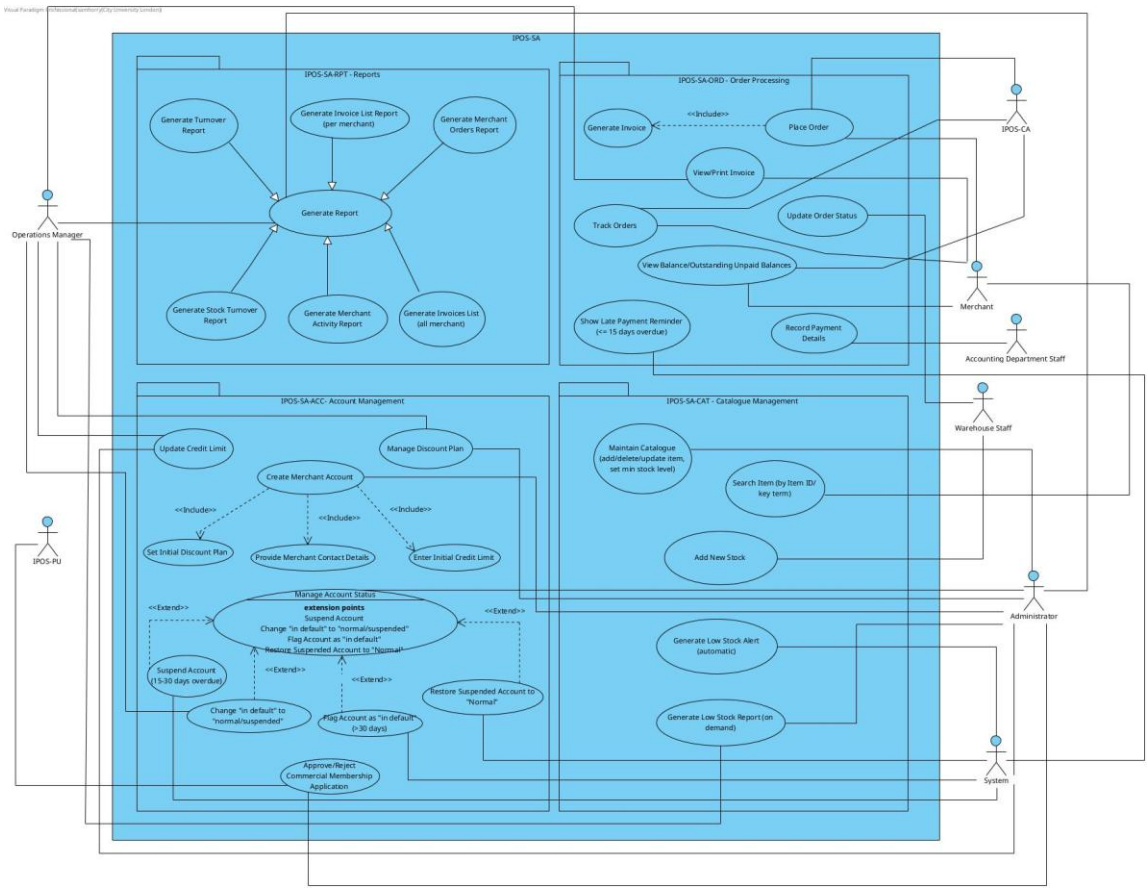


Figure 1. Use case diagram of the InfoPharma Ordering System – Server Application. The use cases cover the required functionality. The actions represent any external entities, such as users or 3rd party systems that may interact with IPOS-SA.

5.2.1.2 Use case specifications (simplified)

UC-SA-ACC-01 — Create Merchant Account

Use case ID: UC-SA-ACC-01	**Use case:** Create Merchant Account
Brief Description: The Administrator creates a new merchant account so that the merchant can access the IPOS-SA catalogue and ordering functions. The account is created and activated only if all mandatory setup information is provided, including contact details, an initial credit limit and an initial discount plan.	
Primary actors: Administrator	
Secondary actors: None.	
Preconditions: IPOS-SA is operational. The Administrator is authenticated and authorised to create merchant accounts.	
Main flow: 1. The Administrator selects the option to create a new merchant account. 2. The system prompts the Administrator to enter the required merchant information. 3. The Administrator provides the merchant contact details (included use case UC-SA-ACC-01a). 4. The Administrator enters the initial credit limit (included use case UC-SA-ACC-01b). 5. The Administrator selects and confirms the initial discount plan (included use case UC-SA-ACC-01c). 6. The system validates that all mandatory information has been provided and is valid. 7. If validation is successful, the system creates and activates the merchant account.	
Extensions: None.	
Inclusions: Provide Merchant Contact Details (UC-SA-ACC-01a); Enter Initial Credit Limit (UC-SA-ACC-01b); Set Initial Discount Plan (UC-SA-ACC-01c).	
Postconditions: The merchant account is created and activated only if all included setup steps are completed successfully; otherwise, the account is not created.	
Alternative flows: A1: The Administrator cancels the process → No account is created. A2: Mandatory data is missing or invalid → The system displays an error message, and the merchant account is not created. The Administrator must correct the information and resubmit the request.	

UC-SA-ACC-01a — Provide Merchant Contact Details (included)

Use case ID: UC-SA-ACC-01a	**Use case:** Provide Merchant Contact Details
Brief Description: As part of creating a merchant account, the Administrator provides the mandatory merchant contact information required before the account can be created and activated.	
Primary actors: Administrator	
Secondary actors: None.	
Preconditions: UC-SA-ACC-01 (Create Merchant Account) has been initiated by the Administrator.	
Main flow: 1. The system prompts the Administrator to enter the required merchant contact details. 2. The Administrator enters and confirms the contact information. 3. The system validates the entered data. 4. If the data is valid, the system stores the contact details for the merchant account being created.	
Extensions: None.	
Inclusions: None.	
Postconditions: The valid merchant contact details are stored as part of the merchant account creation process.	
Alternative flows: A1: The Administrator cancels the process → The contact details are not stored and control returns to UC-SA-ACC-01. A2: Required fields are missing or invalid → The system displays an error message and requests correction. The contact details are not stored until valid data is provided.	

UC-SA-ACC-01b — Enter Initial Credit Limit (included)

Use case ID: UC-SA-ACC-01b	**Use case:** Enter Initial Credit Limit
Brief Description: The Administrator sets the initial credit limit for the merchant account being created.	
Primary actors: Administrator	
Secondary actors: None.	
Preconditions: UC-SA-ACC-01 (Create Merchant Account) has been initiated by the Administrator.	
Main flow: 1. The system prompts the Administrator to enter the initial credit limit. 2. The Administrator enters and confirms the credit limit value. 3. The system validates the entered value. 4. If the value is valid, the system stores the initial credit limit for the merchant account being created.	
Extensions: None.	
Inclusions: None.	
Postconditions: The initial credit limit is stored for the merchant account being created.	
Alternative flows: A1: The Administrator cancels the process → The credit limit is not stored and control returns to UC-SA-ACC-01. A2: The entered credit limit is invalid (e.g., negative value) → The system displays an error message and the credit limit is not stored.	

UC-SA-ACC-01c — Set Initial Discount Plan (included)

Use case ID: UC-SA-ACC-01c	**Use case:** Set Initial Discount Plan
Brief Description: The Administrator assigns an initial discount plan to the merchant account being created. The system supports fixed and flexible discount plans.	
Primary actors: Administrator	
Secondary actors: None.	

Preconditions: UC-SA-ACC-01 (Create Merchant Account) has been initiated by the Administrator.
Main flow: 1. The system prompts the Administrator to select an initial discount plan for the merchant account being created. 2. The system displays the available discount plan types (fixed or flexible). 3. The Administrator selects and confirms the appropriate discount plan. 4. The system validates the selected plan configuration. 5. If valid, the system stores the discount plan for the merchant account being created.
Extensions: None.
Inclusions: None.
Postconditions: The selected discount plan is stored for the merchant account being created.
Alternative flows: A1: The Administrator cancels the process → The discount plan is not stored and control returns to UC-SA-ACC-01. A2: The selected discount plan configuration is invalid → The system displays an error message and the discount plan is not stored.

UC-SA-ACC-02 — Approve/Reject Commercial Membership Application

Use case ID: UC-SA-ACC-02	**Use case:** Approve/Reject Commercial Membership Application
Brief Description: The Administrator reviews commercial membership applications received from IPOS-PU and decides whether to approve or reject them.	
Primary actors: Administrator	
Secondary actors: IPOS-PU	
Preconditions: A commercial membership application has been received from IPOS-PU and is available in the IPOS-SA review queue.	

Main flow: 1. The Administrator selects a pending commercial membership application from the IPOS-SA review queue. 2. The system displays the application details received from IPOS-PU. 3. The Administrator reviews the application information. 4. The Administrator selects either approve or reject. 5. The system records the decision. 6. If approved, the application status is updated accordingly in IPOS-SA.
Extensions: None.
Inclusions: None.
Postconditions: The application is marked as approved or rejected and the decision is recorded in IPOS-SA.
Alternative flows: A1: The Administrator cancels the review process → No decision is recorded. A2: The Administrator selects reject → The system records the rejection decision and updates the application status.

UC-SA-ACC-03 — Update Credit Limit

Use case ID: UC-SA-ACC-03	**Use case:** Update Credit Limit
Brief Description: The Administrator or Operations Manager updates the credit limit for an existing merchant account.	
Primary actors: Administrator; Operations Manager	
Secondary actors: None.	
Preconditions: The merchant account exists. The Administrator or Operations Manager is authenticated and authorised to update credit limits.	
Main flow: 1. The Administrator or Operations Manager selects the merchant account to be updated. 2. The system displays the current credit limit. 3. The actor enters the new credit limit value and confirms the change. 4. The system validates the entered value. 5. If valid, the system updates and stores the merchant's credit limit.	
Extensions: None.	
Inclusions: None.	
Postconditions: The merchant's credit limit is updated and stored in IPOS-SA.	
Alternative flows: A1: The actor cancels the process → The credit limit remains unchanged. A2: The entered credit limit value is invalid → The system displays an error message and the credit limit is not updated.	

UC-SA-ACC-04 — Manage Discount Plan

Use case ID: UC-SA-ACC-04	**Use case:** Manage Discount Plan
Brief Description: The Administrator or Operations Manager updates the discount plan for an existing merchant account. The system supports fixed and flexible (monthly aggregated orders) discount plans.	
Primary actors: Administrator; Operations Manager	
Secondary actors: None.	
Preconditions: The merchant account exists. The Administrator or Operations Manager is authenticated and authorised to manage discount plans.	
Main flow: 1. The Administrator or Operations Manager selects the merchant account to be updated. 2. The system displays the current discount plan. 3. The actor selects the required discount plan type and enters any required plan details. 4. The system validates the entered plan information. 5. If valid, the system stores the updated discount plan for the merchant account.	
Extensions: None.	
Inclusions: None.	
Postconditions: The merchant's discount plan is updated and stored in IPOS-SA.	
Alternative flows: A1: The actor cancels the process → The discount plan remains unchanged. A2: The entered plan information is invalid → The system displays an error message and the discount plan is not updated.	

UC-SA-ACC-05 — Manage Account Status

Use case ID: UC-SA-ACC-05	**Use case:** Manage Account Status
Brief Description: This base use case allows an authorised user to manage the merchant account status. Specific status changes are handled through extension use cases.	
Primary actors: Administrator	
Secondary actors: None.	
Preconditions: A merchant account exists in IPOS-SA.	
Main flow: 1. The Administrator selects the option to manage a merchant account status. 2. The system triggers the relevant extension use case depending on the requested status action.	
Extensions: Suspend Account (15–30 days overdue) (UC-SA-ACC-05a); Flag Account as “in default” (>30 days) (UC-SA-ACC-05b); Restore Suspended Account to “Normal” (UC-SA-ACC-05c); Change “in default” to “normal/suspended” (UC-SA-ACC-05d).	
Inclusions: None.	
Postconditions: The merchant account status is updated according to the executed extension use case.	
Alternative flows: None (extensions handle alternatives).	

UC-SA-ACC-05a — Suspend Account (15–30 days overdue) — extends Manage Account Status

Use case ID: UC-SA-ACC-05a	**Use case:** Suspend Account (15–30 days overdue)
Brief Description: When payment is overdue between 15 and 30 days, IPOS-SA suspends the merchant account so no further orders are accepted.	
Primary actors: System (Automated processes)	
Secondary actors: None.	
Preconditions: The merchant account exists. The payment is overdue for more than 15 days and not more than 30 days.	
Main flow: 1. The system detects that the merchant's payment is overdue for more than 15 days and not more than 30 days. 2. The system changes the account status to suspended. 3. The system prevents the merchant from placing new orders.	
Extensions: None.	
Inclusions: None.	
Postconditions: The merchant account status is set to suspended and the merchant is prevented from placing new orders.	
Alternative flows: A1: If the payment is overdue for 15 days or less → The account is not suspended and the Show Late Payment Reminder (<= 15 days overdue) use case applies (UC-SA-ORD-08). A2: If the payment is overdue for more than 30 days → The account is flagged as "in default" (>30 days) (UC-SA-ACC-05b).	

UC-SA-ACC-05b — Flag Account as “in default” (>30 days) — extends Manage Account Status

Use case ID: UC-SA-ACC-05b	**Use case:** Flag Account as “in default” (>30 days)
Brief Description: If a payment is overdue for more than 30 days, IPOS-SA flags the merchant account as "in default".	
Primary actors: System (Automated processes)	
Secondary actors: None.	
Preconditions: The merchant account exists. The payment is overdue for more than 30 days.	
Main flow: 1. The system detects that the merchant’s payment is overdue for more than 30 days. 2. The system changes the merchant account status to "in default".	
Extensions: None.	
Inclusions: None.	
Postconditions: The merchant account status is set to "in default".	
Alternative flows: None.	

UC-SA-ACC-05c — Restore Suspended Account to “Normal” — extends Manage Account Status

Use case ID: UC-SA-ACC-05c	**Use case:** Restore Suspended Account to “Normal”
Brief Description: If payment is received within 30 days of the payment deadline, the system restores a suspended account to “Normal” so the merchant can resume ordering.	
Primary actors: System (Automated processes)	
Secondary actors: None.	

Preconditions: The account status is suspended. Payment has been received within 30 days of the payment deadline.
Main flow: 1. The system detects that a payment has been received within 30 days of the payment deadline. 2. The system verifies that the account status is suspended. 3. The system changes the account status to “Normal”. 4. The system allows the merchant to resume placing orders.
Extensions: None.
Inclusions: None.
Postconditions: The merchant account status is set to “Normal” and ordering is enabled.
Alternative flows: A1: If payment is not received within 30 days of the payment deadline → The account is not restored to “Normal”.

UC-SA-ACC-05d — Change “in default” to “normal/suspended” — extends Manage Account Status

Use case ID: UC-SA-ACC-05d	**Use case:** Change “in default” to “normal/suspended”
Brief Description: An account that is in default can be changed to “Normal” or “Suspended” only after authorisation, and the Operations Manager performs the status change in IPOS-SA.	
Primary actors: Operations Manager	
Secondary actors: None.	
Preconditions: The merchant account status is “in default”. The Operations Manager is authenticated and authorised to change account status.	
Main flow: 1. The Operations Manager selects the merchant account that is in default. 2. The system displays the current account status. 3. The Operations Manager selects the new status (“Normal” or “Suspended”). 4. The system updates the account status according to the authorised decision.	
Extensions: None.	
Inclusions: None.	
Postconditions: The merchant account status is updated to “Normal” or “Suspended”.	
Alternative flows: A1: The Operations Manager cancels the process → The account status remains “in default”. A2: Authorisation is not granted → The account status remains “in default” and no status change is applied.	

UC-SA-CAT-01 — Maintain Catalogue (add/delete/update item, set min stock level)

Use case ID: UC-SA-CAT-01	**Use case:** Maintain Catalogue (add/delete/update item, set min stock level)

Brief Description: The Administrator maintains the electronic product catalogue by adding, deleting, or updating items and their attributes (e.g., item ID, description, unit price, availability, and minimum stock level).
Primary actors: Administrator
Secondary actors: None.
Preconditions: The Administrator is authenticated and authorised. The product catalogue is available in IPOS-SA.
Main flow: 1. The Administrator selects an option to add, update, or delete a catalogue item, or to set the minimum stock level for an item. 2. The system displays the relevant item details or an empty form for a new item. 3. The Administrator enters or edits the item information and confirms the change. 4. The system validates the entered data. 5. If valid, the system saves the changes to the catalogue.
Extensions: None.
Inclusions: None.
Postconditions: The catalogue item is added, updated, or deleted (or the minimum stock level is updated), and the changes are stored in IPOS-SA.
Alternative flows: A1: The Administrator cancels the action → No catalogue changes are saved. A2: Item data is invalid (e.g., duplicate item ID or invalid values) → The system displays an error message and no changes are saved.

UC-SA-CAT-02 — Search Item (by Item ID / key term)

Use case ID: UC-SA-CAT-02	**Use case:** Search Item (by Item ID / key term)
Brief Description: The Merchant searches the catalogue by item ID and/or key terms and views the matching items.	
Primary actors: Merchant	
Secondary actors: None.	
Preconditions: The merchant account exists and is active. The Merchant is authenticated.	
Main flow: 1. The Merchant enters an item ID or a key term. 2. The system searches the product catalogue. 3. The system displays the matching items.	
Extensions: None.	
Inclusions: None.	
Postconditions: Matching catalogue items are displayed to the Merchant.	
Alternative flows: A1: No matching items are found → The system displays an empty result list (or a "no results" message). A2: The Merchant cancels the search → No results are displayed.	

UC-SA-CAT-03 — Add New Stock

Use case ID: UC-SA-CAT-03	**Use case:** Add New Stock
Brief Description: When new stock is delivered, Warehouse Staff records the additional quantity for a catalogue item, increasing its available stock level.	
Primary actors: Warehouse Staff	
Secondary actors: None.	
Preconditions: The Warehouse Staff member is authenticated and authorised. The catalogue item exists in IPOS-SA.	
Main flow: 1. The Warehouse Staff selects the catalogue item to update. 2. The system displays the current stock level. 3. The Warehouse Staff enters the additional quantity received. 4. The system validates the entered quantity. 5. If valid, the system updates and stores the new stock level.	
Extensions: None.	
Inclusions: None.	
Postconditions: The stock level of the catalogue item is increased and the updated value is stored in IPOS-SA.	
Alternative flows: A1: The Warehouse Staff cancels the action → No stock changes are recorded. A2: The entered quantity is invalid → The system displays an error message and no changes are saved. A3: The selected item does not exist → The system displays an error message and no changes are saved.	

UC-SA-CAT-04 — Generate Low Stock Alert (automatic)

Use case ID: UC-SA-CAT-04	**Use case:** Generate Low Stock Alert (automatic)

Brief Description: The system checks for low-stock conditions (stock level below the configured minimum stock level) and displays a low-stock warning to relevant users when they access IPOS-SA.
Primary actors: System (Automated processes)
Secondary actors: None.
Preconditions: Minimum stock levels are configured for catalogue items.
Main flow: 1. The system checks catalogue items against their configured minimum stock levels. 2. If any item is below its minimum stock level, the system displays a low-stock warning when relevant users access IPOS-SA.
Extensions: None.
Inclusions: None.
Postconditions: If low stock exists, a warning is displayed when relevant users access IPOS-SA.
Alternative flows: A1: No items are below their minimum stock level → No warning is displayed.

UC-SA-CAT-05 — Generate Low Stock Report (on demand

Use case ID: UC-SA-CAT-05	**Use case:** Generate Low Stock Report (on demand)
Brief Description: The Administrator or Operations Manager generates a report listing all catalogue items whose stock level is below their configured minimum stock level.	
Primary actors: Administrator; Operations Manager	
Secondary actors: None.	
Preconditions: The Administrator or Operations Manager is authenticated and authorised. Minimum stock levels are configured for catalogue items.	
Main flow: 1. The Administrator or Operations Manager selects the option to generate the low stock report.	

2. The system retrieves catalogue items whose stock level is below the configured minimum stock level. 3. The system generates and displays the low stock report.
Extensions: None.
Inclusions: None.
Postconditions: The low stock report is generated and displayed.
Alternative flows: A1: The actor cancels the action → No report is generated. A2: No items are below their minimum stock level → The system generates a report with an empty list (or displays a message).

UC-SA-ORD-01 — Place Order

Use case ID: UC-SA-ORD-01	**Use case:** Place Order
Brief Description: The Merchant places a new order by selecting items from the current catalogue and submitting the order. When an order is placed, the system generates an invoice (included use case).	
Primary actors: Merchant	
Secondary actors: IPOS-CA	
Preconditions: The merchant account exists and is active. The Merchant is authenticated. Catalogue items exist.	
Main flow: 1. The Merchant selects items from the catalogue and specifies quantities. 2. The Merchant submits the order. 3. The system records the order. 4. The system triggers invoice generation (included use case: Generate Invoice).	
Extensions: None.	
Inclusions: Generate Invoice (UC-SA-ORD-02) (included).	

Postconditions: The order is recorded. An invoice is generated for the order.
Alternative flows: A1: The merchant account is not active or the credit limit would be exceeded → The order is not recorded and no invoice is generated. A2: The Merchant cancels the order before submission → No order is recorded.

UC-SA-ORD-02 — Generate Invoice (included)

Use case ID: UC-SA-ORD-02	**Use case:** Generate Invoice
Brief Description: The system generates an invoice for an order. The invoice is stored in the system database for further reference and can be displayed on screen or printed.	
Primary actors: System (Automated processes)	
Secondary actors: None.	
Preconditions: An order has been placed.	
Main flow: 1. The system retrieves the order details. 2. The system calculates the invoice total based on the ordered items. 3. The system creates the invoice for the order. 4. The system stores the invoice in the system database.	
Extensions: None.	
Inclusions: None.	
Postconditions: The invoice is created and stored in the system database.	
Alternative flows: A1: Invoice generation fails → The invoice is not created.	

UC-SA-ORD-03 — View / Print Invoice

Use case ID: UC-SA-ORD-03	**Use case:** View / Print Invoice
Brief Description: The Merchant or Operations Manager displays an invoice on screen and/or prints it.	
Primary actors: Merchant; Operations Manager	
Secondary actors: None.	
Preconditions: The invoice exists in the system database. The Merchant or Operations Manager is authenticated and authorised to access the invoice.	
Main flow: 1. The Merchant or Operations Manager selects an invoice to view. 2. The system retrieves the invoice details from the system database. 3. The system displays the invoice. 4. If requested, the system prints the invoice.	
Extensions: None.	
Inclusions: None.	
Postconditions: The invoice is displayed and/or printed.	
Alternative flows: A1: The actor cancels the action → No invoice is displayed or printed. A2: The invoice is not found → No invoice is displayed. A3: The actor is not authorised to access the invoice → Access is denied and no invoice is displayed.	

UC-SA-ORD-04 — Update Order Status

Use case ID: UC-SA-ORD-04	**Use case:** Update Order Status
Brief Description: The Warehouse Staff updates the status of an existing order. If the order status is set to "dispatched", dispatch details are recorded.	
Primary actors: Warehouse Staff	
Secondary actors: None.	
Preconditions: The order exists. The Warehouse Staff member is authenticated and authorised.	
Main flow: 1. The Warehouse Staff selects an existing order. 2. The system displays the current order status and details. 3. The Warehouse Staff selects a new valid status (accepted, being processed, or dispatched). 4. If the selected status is "dispatched", the system prompts for dispatch details. 5. The Warehouse Staff enters the required dispatch details: • who dispatched the order • dispatch date/time • courier used • courier reference number • expected delivery time 6. The system validates the entered data. 7. If valid, the system updates the order status. 8. If the status is "dispatched", the system stores the dispatch details.	
Extensions: None.	
Inclusions: None.	
Postconditions: Order status is updated. If the status is "dispatched", dispatch details are stored in the system database.	
Alternative flows: A1: The Warehouse Staff cancels the action → No status change is recorded. A2: Mandatory dispatch details are missing when setting status to "dispatched" → The status is not updated.	

UC-SA-ORD-05 — Track Orders

Use case ID: UC-SA-ORD-05	**Use case:** Track Orders
Brief Description: The Merchant tracks the status of their orders maintained by IPOS-SA (e.g., accepted, being processed, dispatched).	
Primary actors: Merchant	
Secondary actors: IPOS-CA	
Preconditions: The Merchant is authenticated.	
Main flow: 1. The Merchant requests to view their orders. 2. The system retrieves all orders belonging to the Merchant. 3. The system displays the list of orders with their current status.	
Extensions: None.	
Inclusions: None.	
Postconditions: The current order status information is displayed.	
Alternative flows: A1: No orders exist for the Merchant → The system displays an empty list (or a "no orders" message). A2: The Merchant cancels the action → No order information is displayed.	

UC-SA-ORD-06 — View Balance / Outstanding Unpaid Balances

Use case ID: UC-SA-ORD-06	**Use case:** View Balance / Outstanding Unpaid Balances
Brief Description: The Merchant views their outstanding unpaid balance through IPOS-SA. Payments are performed outside IPOS-SA, and the displayed balance reflects payments recorded in the system.	
Primary actors: Merchant	
Secondary actors: IPOS-CA	
Preconditions: The Merchant account exists and is active. The Merchant is authenticated.	
Main flow: 1. The Merchant requests to view their account balance. 2. The system retrieves the current outstanding balance for the merchant account. 3. The system displays the outstanding unpaid balance.	
Extensions: None.	
Inclusions: None.	
Postconditions: The outstanding balance is displayed.	
Alternative flows: A1: The Merchant cancels the action → No balance information is displayed.	

UC-SA-ORD-07 — Record Payment Details

Use case ID: UC-SA-ORD-07	**Use case:** Record Payment Details
Brief Description: After a merchant makes a payment outside IPOS-SA, Accounting Department Staff manually enters the payment details into IPOS-SA so that the merchant account balance can be updated.	
Primary actors: Accounting Department Staff	
Secondary actors: None.	
Preconditions: The merchant account exists. The payment has been received outside IPOS-SA. The Accounting Department Staff is authenticated and authorised.	
Main flow: 1. The Accounting Department Staff selects the option to record a payment. 2. The system prompts for payment details. 3. The staff member enters the payment information and confirms. 4. The system validates the entered data. 5. If valid, the system records the payment details and updates the merchant account balance.	
Extensions: None.	
Inclusions: None.	
Postconditions: Payment details are recorded and the merchant account balance is updated.	
Alternative flows: A1: The staff member cancels the action → No payment details are recorded. A2: Required payment details are missing or invalid → The system does not record the payment details and the balance is not updated.	

UC-SA-ORD-08 — Show Late Payment Reminder (≤15 days overdue)

Use case ID: UC-SA-ORD-08	**Use case:** Show Late Payment Reminder (≤15 days overdue)
Brief Description: If a merchant is late no more than 15 days, IPOS-SA allows ordering to continue but displays a payment reminder on screen every time the merchant accesses the account.	
Primary actors: System (Automated processes)	
Secondary actors: None.	
Preconditions: The merchant account exists and is active. The merchant's payment is overdue by 15 days or less. The Merchant accesses IPOS-SA.	
Main flow: 1. The system identifies that the merchant meets the late-payment reminder condition (≤15 days overdue). 2. When the merchant accesses IPOS-SA, the system displays a payment reminder.	
Extensions: None.	
Inclusions: None.	
Postconditions: A payment reminder is displayed.	
Alternative flows: A1: If the overdue period exceeds 15 days → The account status is handled according to the account status rules (UC-SA-ACC-05a / 05b). A2: If the payment is received and recorded → The reminder is no longer displayed.	

UC-SA-RPT-00 — Generate Report (parent)

Use case ID: UC-SA-RPT-00	**Use case:** Generate Report
Brief Description: The Operations Manager or Administrator generates a report. Specific report types are modelled as specialised (child) use cases.	
Primary actors: Operations Manager; Administrator	

Secondary actors: None.
Preconditions: Actor is authenticated and authorised.
Main flow: 1. The actor selects the option to generate a report. 2. The system generates and displays the selected report.
Extensions: Specific report types are modelled as specialised (child) use cases.
Inclusions: None.
Postconditions: A report is generated and displayed.
Alternative flows: A1: The actor cancels the action → No report is generated.

UC-SA-RPT-01 — Generate Turnover Report (specialises UC-SA-RPT-00)

Use case ID: UC-SA-RPT-01	**Use case:** Generate Turnover Report
Brief Description: The Operations Manager or Administrator generates a turnover report for a selected period, showing total quantities of goods sold to merchants and total revenue received by InfoPharma Ltd.	
Primary actors: Operations Manager; Administrator	
Secondary actors: None.	
Preconditions: Actor is authenticated and authorised.	
Main flow: 1. Inherits the main flow of UC-SA-RPT-00 (Generate Report). 2. The actor selects the reporting period. 3. The system calculates total quantities sold and total revenue for the selected period. 4. The system generates and displays the turnover report.	
Extensions: None.	

Inclusions: None.
Postconditions: Turnover report is generated and displayed.
Alternative flows: A1: No transactions exist for the selected period → The system displays zero totals or an empty report.

UC-SA-RPT-02 — Generate Merchant Orders Report (specialises UC-SA-RPT-00)

Use case ID: UC-SA-RPT-02	**Use case:** Generate Merchant Orders Report
Brief Description: The Operations Manager or Administrator generates a report listing orders for a selected merchant for a selected period.	
Primary actors: Operations Manager; Administrator	
Secondary actors: None.	
Preconditions: Actor is authenticated and authorised. The selected merchant exists.	
Main flow: 1. Inherits the main flow of UC-SA-RPT-00 (Generate Report). 2. The actor selects the merchant and the reporting period. 3. The system retrieves all orders for the selected merchant in the selected period. 4. The system generates and displays the merchant orders report.	
Extensions: None.	
Inclusions: None.	
Postconditions: The merchant orders report is generated and displayed.	
Alternative flows: A1: No orders exist for the selected merchant in the selected period → The system displays an empty report (or a “no orders” message).	

UC-SA-RPT-03 — Generate Merchant Activity Report (specialises UC-SA-RPT-00)

Use case ID: UC-SA-RPT-03	**Use case:** Generate Merchant Activity Report
Brief Description: The Operations Manager or Administrator generates a merchant activity report for a selected merchant for a selected period.	
Primary actors: Operations Manager; Administrator	
Secondary actors: None.	
Preconditions: Actor is authenticated and authorised. The selected merchant exists.	
Main flow: 1. Inherits the main flow of UC-SA-RPT-00 (Generate Report). 2. The actor selects the merchant and the reporting period. 3. The system retrieves the merchant activity data for the selected merchant and period. 4. The system generates and displays the merchant activity report.	
Extensions: None.	
Inclusions: None.	
Postconditions: The merchant activity report is generated and displayed.	
Alternative flows: A1: No activity exists for the selected merchant in the selected period → The system displays an empty report (or a message).	

UC-SA-RPT-04 — Generate Invoice List Report (per merchant) (specialises UC-SA-RPT-00)

Use case ID: UC-SA-RPT-04	**Use case:** Generate Invoice List Report (per merchant)
Brief Description: The Operations Manager or Administrator generates a report listing invoices raised for a selected merchant in a selected period.	
Primary actors: Operations Manager; Administrator	
Secondary actors: None.	

Preconditions: Actor is authenticated and authorised. The selected merchant exists.
Main flow: 1. Inherits the main flow of UC-SA-RPT-00 (Generate Report). 2. The actor selects the merchant and the selected period. 3. The system retrieves invoices for the selected merchant in the selected period. 4. The system generates and displays the invoice list report.
Extensions: None.
Inclusions: None.
Postconditions: The invoice list report is generated and displayed.
Alternative flows: A1: No invoices exist for the selected merchant in the selected period → The system displays an empty report (or a "no invoices" message).

UC-SA-RPT-05 — Generate Invoices List (all merchant) (specialises UC-SA-RPT-00)

Use case ID: UC-SA-RPT-05	**Use case:** Generate Invoices List (all merchant)
Brief Description: The Operations Manager or Administrator generates a report listing all invoices raised against all merchants for a selected period. The report is displayed on screen and may be printed.	
Primary actors: Operations Manager; Administrator	
Secondary actors: None.	
Preconditions: Actor is authenticated and authorised.	
Main flow: 1. Inherits the main flow of UC-SA-RPT-00 (Generate Report). 2. The actor selects the reporting period. 3. The system retrieves all invoices issued in the selected period. 4. The system generates and displays the invoices list report.	

Extensions: None.
Inclusions: None.
Postconditions: The invoices list report is generated and displayed. It may be printed.
Alternative flows: A1: No invoices exist in the selected period → The system displays an empty report (or a "no invoices" message).

UC-SA-RPT-06 — Generate Stock Turnover Report (specialises UC-SA-RPT-00)

Use case ID: UC-SA-RPT-06	**Use case:** Generate Stock Turnover Report
Brief Description: The Operations Manager or Administrator generates a stock turnover report for a selected period.	
Primary actors: Operations Manager; Administrator	
Secondary actors: None.	
Preconditions: Actor is authenticated and authorised.	
Main flow: 1. Inherits the main flow of UC-SA-RPT-00 (Generate Report). 2. The actor selects the reporting period. 3. The system retrieves stock turnover data for the selected period. 4. The system generates and displays the stock turnover report.	
Extensions: None.	
Inclusions: None.	
Postconditions: The stock turnover report is generated and displayed.	
Alternative flows: A1: No stock turnover data exists in the selected period → The system displays an empty report (or totals shown as zero).	

5.2.1.3 Prioritised List of use cases

ACC — Account Management

Use case ID	Use case name	UC priority	Justification
UC-SA-ACC-01	Create Merchant Account	Essential	Merchant account creation and activation is a prerequisite for granting controlled access to IPOS-SA. The account must be fully configured (contact details, credit limit and discount plan) before catalogue access and ordering functions are enabled.
UC-SA-ACC-01a	Provide Merchant Contact Details (included by UC-SA-ACC-01)	Essential	Merchant contact details are mandatory to create and activate a merchant account and to enable account communication and identification.
UC-SA-ACC-01b	Enter Initial Credit Limit (included by UC-SA-ACC-01)	Essential	IPOS-SA enforces credit control rules, therefore the initial credit limit must be defined during account creation to enable ordering within the approved credit limit.
UC-SA-ACC-01c	Set Initial Discount Plan (included by UC-SA-ACC-01)	Essential	Mandatory setup information required for creating and activating a merchant account.
UC-SA-ACC-02	Approve/Reject Commercial Membership Application	Essential	Commercial membership applications submitted via IPOS-PU must be reviewed and approved or rejected before applicants are granted access to IPOS-SA.
UC-SA-ACC-03	Update Credit Limit	Important	Needed to manage merchant credit limits after accounts are created.
UC-SA-ACC-04	Manage Discount Plan	Important	Required to update a merchant's discount plan after account creation, ensuring that the assigned discount plan reflects current arrangements.
UC-SA-ACC-05	Manage Account Status	Essential	Enforces overdue-payment rules and enables the system to control whether merchants can continue ordering.
UC-SA-ACC-05a	Suspend Account (15–30 days overdue) (extends UC-SA-ACC-05)	Essential	Required to suspend the merchant account when payment is overdue between 15 and 30 days, preventing further orders as specified in the overdue-payment policy.
UC-SA-ACC-05b	Flag Account as "in default" (>30 days) (extends UC-SA-ACC-05)	Essential	Required to mark a merchant account as "in default" when the payment delay exceeds 30 days, in accordance with the defined overdue-payment policy.
UC-SA-ACC-05c	Restore Suspended Account to "Normal" (extends UC-SA-ACC-05)	Essential	Required to restore a suspended merchant account to "normal" when payment is received within 30 days of the payment deadline, allowing the merchant to resume ordering.

UC-SA-ACC-05d	Change “in default” to “normal/suspended” (extends UC-SA-ACC-05)	Essential	Required to allow the Operations Manager to change an account from “in default” to “normal” or “suspended” following authorised intervention.

CAT — Catalogue Management

Use case ID	Use case name	UC priority	Justification
UC-SA-CAT-01	Maintain Catalogue (add/delete/update item, set min stock level)	Essential	Catalogue data (including item details and minimum stock levels) is required for ordering, stock control, low-stock monitoring and reporting. Without maintaining the catalogue, core system functionality cannot operate.
UC-SA-CAT-02	Search Item (by Item ID / key term)	Important	Merchants need to locate catalogue items before placing orders; this functionality enables efficient product search.
UC-SA-CAT-03	Add New Stock	Essential	Stock availability must be maintained in the system and recording new stock is necessary to ensure accurate stock control.
UC-SA-CAT-04	Generate Low Stock Alert (automatic)	Important	Required to monitor minimum stock levels and notify relevant users when stock falls below the defined threshold.
UC-SA-CAT-05	Generate Low Stock Report (on demand)	Important	Provides visibility of catalogue items below their configured minimum stock level and supports stock monitoring and replenishment planning. However, it is not required for placing or processing orders.

ORD — Order Processing

Use case ID	Use case name	UC priority	Justification
UC-SA-ORD-01	Place Order	Essential	Core operational function that enables merchants to submit purchase orders for catalogue items. Without this use case, the system cannot support its primary ordering workflow defined for IPOS-SA.
UC-SA-ORD-02	Generate Invoice (included by UC-SA-ORD-01)	Essential	Invoices must be generated for accepted orders and stored in the system for reference.
UC-SA-ORD-03	View / Print Invoice	Important	Provides access to stored invoices, allowing authorised users to view and print invoice records for reference.

UC-SA-ORD-04	Update Order Status	Essential	Order status must be maintained and updated by Warehouse Staff so that orders can be tracked and kept up to date in the system.
UC-SA-ORD-05	Track Orders	Essential	Merchants must be able to track the status of their orders as maintained in the system.
UC-SA-ORD-06	View Balance / Outstanding Unpaid Balances	Essential	Merchants must be able to view their outstanding unpaid balance recorded in the system, as account status rules depend on payment information.
UC-SA-ORD-07	Record Payment Details	Essential	Accounting Department Staff must record received payments in the system so the merchant's account balance is updated and late-payment account status rules can be applied correctly.
UC-SA-ORD-08	Show Late Payment Reminder (≤15 days overdue)	Essential	The system must display a reminder on screen when a merchant is up to 15 days late with payment while still allowing ordering to continue.

RPT — Reports

Use case ID	Use case name	UC priority	Justification
UC-SA-RPT-00	Generate Report (parent)	Important	Provides the base "Generate Report" function used by the specialised report use cases. It supports management reporting but is not required to place orders or generate invoices.
UC-SA-RPT-01	Generate Turnover Report (specialises UC-SA-RPT-00)	Important	Provides management insight into quantities sold and revenue received for a selected period. It supports financial and operational oversight but is not part of the core ordering and payment-processing workflow.
UC-SA-RPT-02	Generate Merchant Orders Report (specialises UC-SA-RPT-00)	Important	Generates a report listing orders for a selected merchant and period. It supports operational oversight and analysis at merchant level but is not part of the core ordering and payment-processing workflow.
UC-SA-RPT-03	Generate Merchant Activity Report (specialises UC-SA-RPT-00)	Good to have	Provides a detailed report of a selected merchant's activity for a selected period.
UC-SA-RPT-04	Generate Invoice List Report (specialises UC-SA-RPT-00)	Important	Provides a report listing invoices for a selected merchant and period.
UC-SA-RPT-05	Generate Invoices List (specialises UC-SA-RPT-00)	Important	Provides a report listing invoices for all merchants for a selected period. It supports overall financial monitoring but is not required for core ordering and payment-processing functions.

UC-SA-RPT-06	Generate Stock Turnover Report (specialises UC-SA-RPT-00)	Good to have	Useful for stock analysis and management insight but not required for maintaining stock levels or core ordering operations.

6. Anticipated Evolution of System Requirements

We expect the IPOS-SA prototype to be developed and deployed rapidly to demonstrate core functionality to InfoPharma Ltd. However, as the system transitions from a prototype to a full production application, operational experience and enterprise scaling may necessitate several changes. The anticipated evolution of the system's requirements is summarized below.

6.1 Integration with Online Banking

Currently, the prototype requires the Accounting Department to manually enter payment details (such as cheques or bank transfers) into the system to clear a merchant's balance. In the full production version, payment processing is anticipated to be directly integrated with on-line banking, allowing merchants to make seamless, automated payments. This will require the development of secure API connections to external banking providers.

6.2 Enterprise IT Administration

For the prototype phase, security and logging constraints have been relaxed to focus on core logic. However, the final production version of IPOS-SA will be centrally administered by the IT department of InfoPharma Ltd, so we anticipate the requirements might evolve to include system logging and security protocols.

7 High Level Design

This section includes the high-level design of the IPOS software, this includes the subsystems which is made up of communication interfaces between the subsystems. The subsystems mentioned are:

7. 1 IPOS-SA (Server Application)

Server Application subsystem provides the business logic and data storage for the InfoPharma Ordering System, this allows the system to process orders, authenticate merchants and maintain the product catalogue and account balances.

7.2 IPOS-CA (Client Application)

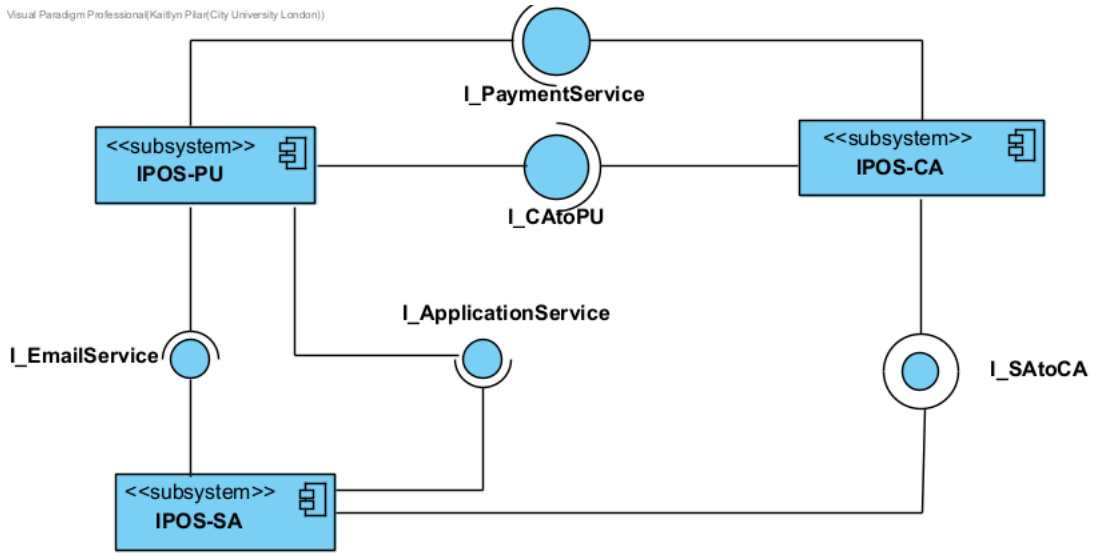
Client application subsystem allows pharmacy merchants to manage their shop’s inventory and purchases made. This connects to the server to place orders and interacts with

7.3 IPOS-PU (Public Online Platform)

The Public Online Platform is a web portal which captures commercial applications from pharmacies and implements payment systems and manages communications by providing email and broadcast services to the other subsystem.

7.4 Software Architecture

Modelled as a Component Diagram, developed with Visual Paradigm in **Figure 2.**



The subsystems are modelled as components with the stereotype: <<subsystem>>

Figure 2 - The communication between each subsystem is set through assembly connectors. Based on the architecture diagrams the systems using these APIs:

I_SAtoCA: Provided by IPOS-SA and required by IPOS-CA, the interfaces allow the CA to access the central product catalogue and submit orders.

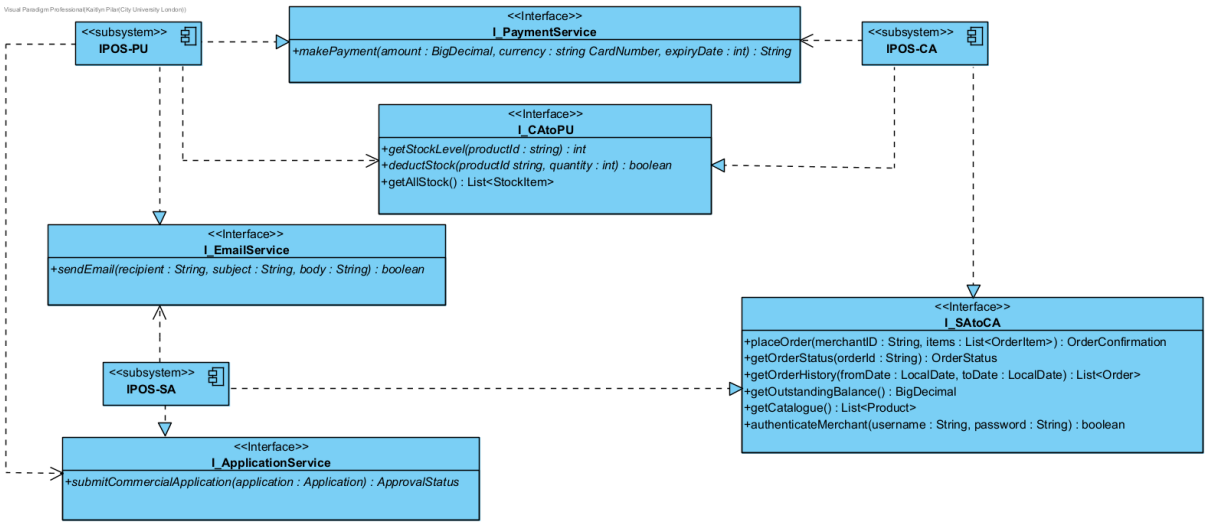
I_ApplicationService: Provided by IPOS-SA and requires IPOS-PU, which allows the online portal to pass membership applications to central server for approval.

I_EmailService: Provided by IPOS-PU and required by IPOS-SA, allows the server to trigger emails which are sent automatically without managing the SMTP integration.

I_PaymentService: This is handled between portals and the client applications to process transactions when orders are made.

I_CAtoPU: Synchronises data such as online sales or updating pharmacy stock levels.

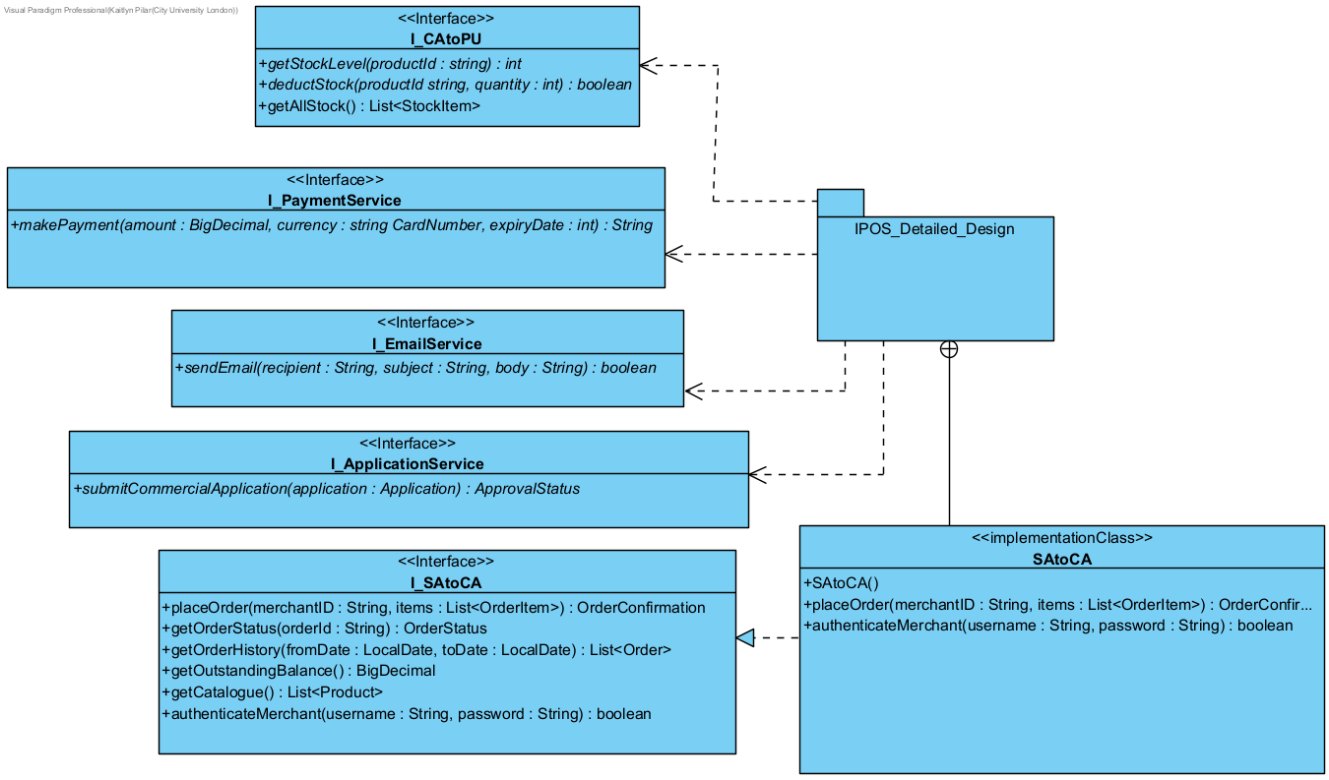
Figure 3



Commentary

In figure 3, the architectural diagrams present a system each subsystem provides dedicated APIs to handle specific domains of the business logic. An example is IPOS-SA act as the provide for I_SAtoCA and I_ApplicationService ensuring any database changes are controlled by the Server Application. On the other hand, IPOS-SA is a dependency on I_EmailService, handling and sending off external communication responsibilities to the portal. This module ensures any changes made will not require code with the core IPOS-SA database logic.

Figure 4



Commentary

In Figure 4 the diagram shows the interface IPOS-SA provides the interface a minimal version of the implementation class SAtoCA. This shows the relationships between the interfaces and the classes showing where the code for the APIs will reside.

We use IPOS_Detailed_Design as a place holder. The content classes will deliver core function by the IPOS and will be developed later as part of the detailed design and implementation phases.

Additionally, the diagram highlights required interfaces such as I_EmailService, the dependencies between the IPOS_DETAILED_DESIGN package and this required interface indicates some classes will make calls to methods in the interface. The portal subsystem will be providing an implementation for methods for the interface.

7.5 Source Code for the Subsystem Interfaces

7.5.1 Overview

This section presents the Java source code generated from the Design Class Diagram fragment for the IPOS-SA subsystem (Team A).

The design fragment includes:

- The subsystem interface(s) provided by IPOS-SA
- The implementation class that realises the interface(s)
- Method signatures with full parameter types and return types
- Constructors for the implementation class

The source code was generated from the UML model using Visual Paradigm. The generated classes represent the formal contract between IPOS-SA and the other subsystems and allow interface-level testing to be performed prior to full implementation.

7.5.2 Interface: I_ApplicationService

```
package IPOS_Detailed_Design;

public interface I_ApplicationService {

	/**
	 *
		* @param application
```

```
         * @param taxID
         * @param address
         */
        abstract ApprovalStatus submitCommercialApplication(Application application);

}
```

Description

The I_ApplicationService interface defines the service boundary for handling commercial applications submitted to IPOS-SA.

The interface supports the submission of commercial registration requests via a single operation:

- submitCommercialApplication(Application application)

The method returns an ApprovalStatus value, representing the outcome of the application processing workflow. This separation ensures that application-handling logic is encapsulated behind a service interface and can be implemented, validated, and tested independently of other subsystems.

7.5.3 Interface: I_CAtoPU

```
package IPOS_Detailed_Design;

public interface I_CAtoPU {

        /**
         *
         * @param productId
         */
        abstract int getStockLevel(string productId);

        /**
         *
         * @param productId
         * @param quantity
         */
        abstract boolean deductStock(string productId, int quantity);

        List<StockItem> getAllStock();

}
```

Description

The I_CAtoPU interface defines a contract for stock-related interaction between subsystems.

The interface supports:

- Checking the current stock level for a specific product (getStockLevel)
- Deducting stock after a confirmed purchase (deductStock)

- Retrieving a full list of stock items (getAllStock)

This interface enables stock-level operations to remain encapsulated within the owning subsystem while exposing only required operations to external components through controlled method calls.

7.5.4 Interface: I_EmailService

```
package IPOS_Detailed_Design;

public interface I_EmailService {

	/**
	 *
	 * @param recipient
	 * @param subject
	 * @param body
	 */
	abstract boolean sendEmail(string recipient, string subject, String body);

}
```

Description

The I_EmailService interface defines the abstraction for sending email notifications.

The interface provides a single method:

- sendEmail(...) which returns a boolean to indicate success or failure.

This design allows the subsystem to trigger messaging operations without direct dependency on the underlying email provider mechanism. It also supports testing using stub or mock implementations while maintaining a consistent call pattern for notification delivery.

7.5.5 Interface: I_PaymentService

```
package IPOS_Detailed_Design;

public interface I_PaymentService {

	/**
	 *
	 * @param amount
	 * @param currency
	 * @param cardNumber
	 * @param expiryDate
	 */
	abstract String makePayment(BigDecimal amount, string currency, string cardNumber, int
expiryDate);

}
```

Description

The I_PaymentService interface defines an abstraction for performing payment transactions.

The interface contains a single operation:

- makePayment(...) returning a String, which may represent a payment reference, receipt identifier, or transaction status code.

This interface separates payment processing concerns from the core subsystem logic and enables payment processing to be implemented using either a simulated transaction flow (prototype stage) or a real payment provider integration.

7.5.6 Interface: I_SAtoCA

```
package IPOS_Detailed_Design;

public interface I_SAtoCA {

        /**
         *
         * @param fromDate
         * @param toDate
         */
        List<Order> getOrderHistory(LocalDate fromDate, LocalDate toDate);

        List<Product> getCatalogue();

        /**
         *
         * @param orderId
         */
        OrderStatus getOrderStatus(string orderId);

        BigDecimal getOutstandingBalance();

        /**
         *
         * @param itemID
         */
        int checkStock(String itemID);

        /**
         *
         * @param merchantID
         * @param items
         */
        void submitOrder(String merchantID, Map<String, Integer> items);

        /**
         *
         * @param username
         * @param password
         */
        boolean authenticateMerchant(string username, string password);

        /**
         *
         * @param merchantID
         * @param items
         */
        OrderConfirmation placeOrder(string merchantID, List<OrderItem> items);
```

```
}
```

Description

The I_SAtoCA interface defines the service contract provided by IPOS-SA to IPOS-CA.

The interface encapsulates the server-side operations required by the merchant subsystem, including:

- Retrieving catalogue data (getCatalogue)
- Placing and submitting orders (placeOrder, submitOrder)
- Checking stock availability (checkStock)
- Authenticating merchant users (authenticateMerchant)
- Tracking orders and account balances (getOrderStatus, getOrderHistory, getOutstandingBalance)

This interface acts as a formal integration boundary, ensuring that cross-subsystem interaction occurs only through explicitly declared methods rather than direct access to subsystem data stores.

7.5.7 Implementation Class: SAtoCA

```
package IPOS_Detailed_Design;

import Design_Model.IPOS_Detailed_Design.I_ServerAPI.*;

public class SAtoCA implements submitOrder, I_SAtoCA {

        public SAtoCA() {
                throw new UnsupportedOperationException();
        }

        /**
         *
         * @param merchantID
         * @param items
         */
        public OrderConfirmation placeOrder(string merchantID, List<OrderItem> items) {
                throw new UnsupportedOperationException();
        }

        /**
         *
         * @param username
         * @param password
         */
        public boolean authenticateMerchant(string username, string password) {
                throw new UnsupportedOperationException();
        }

        public BigDecimal getOutstandingBalance() {
                throw new UnsupportedOperationException();
        }
```

```
        /**
         *
         * @param orderId
         */
        public OrderStatus getOrderStatus(String orderId) {
                throw new UnsupportedOperationException();
        }

        public List<Product> getCatalogue() {
                throw new UnsupportedOperationException();
        }

        /**
         *
         * @param fromDate
         * @param toDate
         */
        public List<Order> getOrderHistory(LocalDate fromDate, LocalDate toDate) {
                throw new UnsupportedOperationException();
        }

}
```

Description

The SAtoCA class represents the implementation class for the I_SAtoCA interface.

The class includes:

- A constructor (SAtoCA())
- Stubbed method bodies that throw UnsupportedOperationException

The stubbed structure provides a validated interface layer and a stable implementation shell which can be incrementally extended during later stages of development. This supports early validation of integration structure and allows interface-level test cases to be designed prior to completing full business logic and database functionality.

7.5.8 Summary

The interfaces presented in this section define the integration boundaries and service contracts of the subsystem. The generated Java source code ensures that the implementation structure matches the design model and establishes a foundation for incremental development and interface-based testing.

7.6 Testing the communications between subsystems

This section defines the interface level test cases to validate communication between IPOS_SA and the other subsystems.

Testing is performed at API contract level prior to full implementation of business logic. Each test verifies correct handling of calid and invalid outputs and ensures that's integration boundaries behave as specified. The different testID letters identify which subsystem interface tests are being done. CA- Interface: I_SAtoCA, A- Interface: I_ApplicationService, PU- Interface: I_CatoPU, P- Interface: PaymentService.

Method	Test ID	Parameter Values	Comment	Expected Outcome
authenticateMerchant(String username, String password): boolean	1CA	Username ="merchant1" Password="correctPass"	Valid registered merchant	Return true
authenticateMerchant(String username, String password): boolean	2CA	Username="merchant1" Password="wrongPass"	Incorrect Password	Return false
authenticateMerchant(String username, String password): boolean	3CA	Username="unknownUser" Password="anyPass"	Merchant doesn't exist	Return false
authenticateMerchant(String username, String password): boolean	4CA	Username="" Password=""	Empty Credentials	Return False
submitOrder(String merchantID, Map<String, Integer> items) : boolean	5CA	merchantID = "M001" items = {("P01", 5)}	Valid merchant and stock available	Return true
submitOrder(String merchantID, Map<String, Integer> items) : OrderConfirmation	6CA	merchandID="M001" items={("P01", 9999)}	Quantity exceeds stock	Return false
submitOrder(String merchantID, Map<String, Integer> items) : OrderConfirmation	7CA	merchandID="InvalidID" items={("P01", 5)}	Merchant doesn't exist	Return false

submitOrder(String merchantID, Map<String, Integer> items) : OrderConfirmation	8CA	merchandID=”M001” items={}	Empty Order	Return false
checkStock(String itemID) : int	9CA	itemID = “P01”	Item exists	Return stock quantity (e.g., 50)
checkStock(String itemID) : int	10CA	itemID = “invalid item”	Item does not exist	Return 0 or error indicator
getOrderHistory(LocalDate fromDate, LocalDate toDate) : List	11CA	Valid Date range	Order exist in range	Return List<Ordeer>
getOrderHistory(LocalDate fromDate, LocalDate toDate) : List	12CA	Valid Date range	No order exist	Return empty list
getOrderHistory(LocalDate fromDate, LocalDate toDate) : List	13CA	fromDate>toDate	Invalid range	Return error date range invalid.
getOrderStatus(String orderId) : OrderStatus	14CA	orderID=”validID”	Valid order ID	Return correct Order Status
getOrderStatus(String orderId) : OrderStatus	15CA	orderID = “invalidID”	Invalid order ID	Return error InvalidID
getOutstandingBalance() : BigDecimal	16CA	Balance = “160.50”	Outstanding balance exists	Return positive value
getOutstandingBalance() : BigDecimal	17CA	Balance=”0.00”	No balance due	Return 0.00
submitCommercialApplication (Application application) : ApprovalStatus	18A	Valid Application Object	All required fields populated correctly	Returns ApprovalStatus.Approved
submitCommercialApplication (Application application) : ApprovalStatus	19A	Application with missing required fields	Incomplete data	Returns ApprovalStatus.Rejected
submitCommercialApplication (Application application) : ApprovalStatus	20A	Null application object	Invalid input	Returns ApprovalStatus.Rejected
getStockLevel(String productId) : int	21PU	productId = "P001"	Product exist	Return stock quantity (e.g., 50)
getStockLevel(String productId) : int	22PU	productId = "INVALID"	Product does not exist	Return 0 or error
deductStock(String productId, int quantity) : boolean	23PU	productId = "P001", quantity = 5	Stock sufficient	Return true
deductStock(String productId, int quantity) : boolean	24PU	productId = "P001", quantity = 9999	Insufficient stock	Returns false

deductStock(String productId, int quantity) : boolean	25PU	productId = "INVALID", quantity = 5	Product does not exist	Return false
deductStock(String productId, int quantity) : boolean	26PU	quantity = -1	Invalid quantity	Return false
getAllStock() : List<StockItem>	27PU	Valid stock items	Stock items exist	Return populated list
getAllStock() : List<StockItem>	28PU	Invalid stock items	No stock items	Return empty list
sendEmail(String recipient, String subject, String body) : boolean	29PU	Valid email address	Proper format	Return true
sendEmail(String recipient, String subject, String body) : boolean	30PU	Invalid email format ("abc@")	Incorrect format	Return false
sendEmail(String recipient, String subject, String body) : boolean	31PU	Empty recipient	Missing required field	Return false
sendEmail(String recipient, String subject, String body) : boolean	32PU	Empty subject/body	Incomplete messege	Return false
makePayment(BigDecimal amount, String currency, String cardNumber, int expiryDate) : String	33P	Valid amount, valid card, future expiry	Valid transation	Return transaction reference ID
makePayment(BigDecimal amount, String currency, String cardNumber, int expiryDate) : String	34P	Amount = 0	Invalid payment amount	Return failure code
makePayment(BigDecimal amount, String currency, String cardNumber, int expiryDate) : String	35P	Invalid card number	Card validation fails	Return failure code
makePayment(BigDecimal amount, String currency, String cardNumber, int expiryDate) : String	36P	Expire card date	Expiry in past	Return failure code

8. Appendices

8.1 Initial Statement of Requirements by the Client

Your company has been invited to develop an "IT system to assist with the distribution of pharmaceutical goods" for InfoPharma Ltd, a major pharmaceutical distributor. InfoPharma intend to optimise their interaction with their clients, retail merchants (individuals running own chemist shop and large retailers with chains of stores, e.g. Boots and Superdrags), who sell drugs and other pharmaceutical goods to the members of the public. The new system will also allow small retailers to sell pharmaceutical goods online.

Currently, InfoPharma communicate with merchants by distributing a catalogue of goods available to the interested parties. The catalogue is printed monthly, but in case of major changes of prices special editions can also be printed between the regular updates. Orders are taken by using conventional means such as phone and more recently – e- mail.

Maintaining this way of interacting with the client base is not justified any longer since most of the retailers are already online and have converted parts of their business to 'paperless' technology. Deploying a B2B suite of software applications seems long overdue. InfoPharma want a smooth transition from the current catalogue-based operation to operation relying on a new software order tracking system (InfoPharma Ordering System, IPOS). They would like to have in place for a few months the current operation (catalogue and orders by phone/email) together with IPOS which would allow retailers to have access to the most up to date catalogue and easy ordering from a desktop PC. In addition, in order to motivate the clients (mainly the pharmacy shop owners) to switch to IPOS quickly, they would like IPOS to include extensive functionality which can be useful to the retailers in their work with their customers: controlling the local stock availability in shops, maintaining customers database and ability to provide discounts to regular customers. IPOS will also allow the shop owner to offer their products online.

InfoPharma Ltd. may decide to offer your company the development of the production version of IPOS and a maintenance contract for it, if you do a good job with the prototype of IPOS.

Your consultant has won the contract from InfoPharma's head office to assess their requirements and produce a project plan for the implementation of one system for one Pharmacy. Your team has been tasked with doing the job. She/he asks you to set up an appointment for an interview with Mr. Lancaster, the founder and CEO at InfoPharma Ltd. Mr Lancaster has agreed to provide clarification on the requirements for the new system.

If the project plan, the accompanying schedule and the requirements documents are accurate, and the cost is acceptable you will be invited to complete the contract and implement the prototype.

Initial Requirement

The InfoPharma Ordering System (IPOS) will keep track of the orders of pharmaceutical

goods made by merchants to InfoPharma Ltd and will assist the merchants' sales to consumers.

IPOS includes three subsystems:

i) A server application (IPOS-SA), which in the fully functional production version will be administered by the IT department of InfoPharma Ltd.

ii) A client application (IPOS-CA), a copy of which will be run by the merchant on a desktop PC in the merchant's Pharmacy shop.

iii) An online platform (IPOS-PU) to be used by a merchant for direct sales of pharmaceutical goods to the public. The platform will allow the merchants to run advertisement/promotion campaigns with new products with discounts.

In the prototype of IPOS the three subsystems should be implemented as separate desktop applications able to interact with one another as necessary to meet the requirements. For instance, online sales (via IPOS-PU) should be linked to the inventory of the merchant: successful online purchases should be deducted from the stock of goods available at merchant's shop. Likewise, orders from IPOS-SA and delivered to a merchant, should be easy to record in IPOS-CA and increase the merchant's stock. Finally, promotion campaigns run via IPOS-PU may affect the cost of selling goods online (at reduced price), but not the cost of the same goods if sold in the pharmacy shops.

8.2 Hardware (Minimal and Optimal Configuration)

8.2.1 Minimal hardware configuration

The minimal hardware configuration for the development of IPOS consists of a laptop or desktop computer, with a multi-threaded 4 core CPU, at least 8GB of RAM and at least 50GB of free hard-drive space for the software programs and data to be stored in a running database server such as MySQL.

8.2.2 Optimal hardware configuration

The optimal hardware configuration, that is recommended for the development of IPOS consists of a laptop or desktop PC, with a multi-threaded 8 core CPU, at least 16GB of RAM, ideally more, and at least 100+GB of free hard-drive space for the software and data to be stored on a database server such as MySQL. This optimal configuration should allow for the best stability when developing and debugging the program, as well as when running the database server locally to avoid system lock-ups.